

Monte Carlo Heuristic + Expectiminimax Agent – Full Documentation

This document provides a complete, clear explanation of:

1. **Monte Carlo Hand Strength Simulation**
2. **Balanced Heuristic Evaluation**
3. **Expectiminimax Algorithm for Poker**
4. **How these components work together to create a strategic poker AI**

1. Monte Carlo Simulation (simulate_hand_strength)

Monte Carlo simulation is used to estimate the probability that the AI wins a hand. Because poker has incomplete information (unknown opponent cards, unknown future community cards), exact evaluation is impossible during gameplay.

Monte Carlo approximates win probability through random sampling.

Step-by-Step Logic

Step A — Shuffle the remaining deck

We take all undealt cards and randomly shuffle them. These cards represent all possible unknown information.

Step B — Complete the community board

If the board has fewer than 5 cards:

- Add random cards from the simulation deck until the board reaches 5 cards.

Step C — Assign opponent hole cards

Two random cards are given to the opponent from the simulation deck.

Step D — Evaluate both hands

Using the PokerHandEvaluator:

- Compare the AI full hand vs opponent full hand.
- Record win / tie / loss.

Step E — Repeat many times

Run between **200–5000 simulations**.

Step F — Estimate win probability:

...

`win_rate = (wins + 0.5 * ties) / valid_simulations`

...

This produces a **statistical estimate** of how strong the AI hand is on average.

2. Balanced State Heuristic (evaluate_state_heuristic)

This heuristic takes the Monte Carlo win probability and applies poker strategy logic to produce a single "state value".

Higher value → more aggressive actions

Lower value → fold/check logic

The heuristic uses:

(A) Hand Strength (MAIN FACTOR)

...

hand_strength_value = win_prob * 100

...

Bonuses:

- +40 for win_prob > 0.75

- +20 for win_prob > 0.65

Penalties:

- -60 for win_prob < 0.35

- -30 for win_prob < 0.45

(B) Expected Pot Value

...

expected_pot_value = win_prob * pot

final_pot_value = expected_pot_value * 1.5

...

(C) Calling Cost Penalty

...

penalty = amount_to_call * 1.8

...

Bad pot-odds penalty if:

...

amount_to_call / pot > 0.5 → extra -40

...

(D) Position Bonus (small)

Players closer to button get +2 per positional advantage.

(E) Stack Factor

Based on stack-to-pot ratio:

- >10 → +15

- <3 → -20

- <2 → -40

Final Formula:

...

value =

expected_pot_value * 1.5 +

hand_strength_value +

position_bonus +

stack_factor -

amount_to_call * 1.8

...

This value becomes the AI's "score" for the current game state.

3. Expectiminimax Algorithm

Expectiminimax is a decision-making algorithm designed for games with:

- **uncertainty (random events)** → chance nodes
- **opponents** → min nodes
- **our AI** → max nodes

Poker fits perfectly:

- Chance = card dealing
- Min = opponents
- Max = AI

The algorithm simulates possible future actions and card outcomes to decide the best move.

3.1 Node Types

MAX Node (AI Turn)

AI chooses action that **maximizes** its future value.

MIN Node (Opponent Turn)

Opponent chooses action that **minimizes** AI's future value.

CHANCE Node (Cards Being Dealt)

Random card draws produce **expected values** across multiple sampled outcomes.

3.2 Tree Search Flow

Search depth (e.g., 3) controls how far ahead the AI looks.

At each node:

1. Clone game state
2. Apply action or deal cards
3. Evaluate next node type
4. Return the best/worst/average value

3.3 Terminal Evaluation

Two scenarios end the branch:

A. Fold

If AI is last active player:

- AI wins entire pot

If AI folded earlier:

- AI loses invested chips

B. Showdown

Use `**evaluate_state_heuristic()**` to compute exact value of terminal showdown.

4. CHANCE NODE (Card Dealing)

Chance node logic:

Preflop → deal 3 cards (flop)

Flop → deal 1 card (turn)

Turn → deal 1 card (river)

The agent:

1. Samples 30 possible combinations
2. Applies card(s) to a cloned state
3. Evaluates value using recursive expectiminimax
4. Averages results

...

`expected_value = sum(sample_values) / num_samples`

...

This models the randomness of real Texas Hold'em accurately.

5. How All Components Work Together

The full architecture:

(1) Monte Carlo Simulation

Gives AI *probability-based hand strength*.

Example:

"AI has 68% chance of winning given its cards and board."

(2) Balanced Heuristic

Transforms win probability + table conditions into a **numeric state score**.

Example:

"With stack sizes, pot odds, and position, this hand is worth +54."

(3) Expectiminimax Search

Looks ahead multiple rounds of:

- betting
- card dealing
- opponent reactions

It uses:

- Monte Carlo → for probability estimation
- Heuristic → for state evaluation
- Search tree → for move selection

(4) Final Decision

Expectiminimax Agent returns the best action:

- fold
- check
- call
- raise X amount

Based on combined:

- probability
- strategy
- game-tree lookahead

6. Summary

This AI poker system is composed of three powerful and complementary components:

✓ Monte Carlo Simulation

Estimates real winning chances under uncertainty.

✓ Balanced Heuristic

Applies strategic poker logic (pot odds, stack depth, position).

✓ Expectiminimax

Simulates the game tree with randomness and opponent behavior.

When combined:

- The AI understands statistical hand value
- Adjusts for real strategic factors
- Predicts and counters opponent actions
- Handles random card dealing
- Makes the best possible action each turn

This results in a ****smart, conservative, and human-like poker-playing AI****.
